

Type 'help', 'copyright', 'credits' or 'license' for more information.

```
>>> import dbus
>>> bus = dbus.SessionBus()
>>>
```

While a `SystemBus`, on the other hand, can be created by simply replacing the `dbus.SessionBus()` element in the above code to `dbus.SystemBus()`:

```
>>> bus = dbus.SystemBus()
```

Every application that intends to share its objects and methods are started as D-Bus services. A D-Bus enabled application exports its objects with their functionalities as methods that other applications can use. By connecting to the corresponding bus and the application object, the application's functionalities can be accessed from other applications.

We use an addressing method to identify each application and its functionalities—reversed domain name addressing. For example, `NetworkManager` is addressed as `org.freedesktop.NetworkManager`, `Pidgin` as `org.gnome.Pidgin`, etc.

Each of the applications can export numerous objects and functions—that is, `NetworkManager` has got different parameters such as 'if network is up or down', 'the current active wifi profile', etc.

Proxy objects and interfaces

The term 'proxy objects' refers to objects that point to remote applications and are accessed through D-Bus session. Let's explore how to create proxy objects.

To obtain a proxy object, call the `get_object` method on the bus. For example, `NetworkManager` has the well-known name `org.freedesktop.NetworkManager` and exports an object whose object path is `/org/freedesktop/NetworkManager`, plus an object per network interface at object paths like `/org/freedesktop/NetworkManager/Devices/wlan0`.

```
>>> import dbus
>>> bus = dbus.SystemBus()
>>> proxy_object = bus.get_object('org.freedesktop.NetworkManager', '/org/freedesktop/NetworkManager')
```

The format of the parameters for `get_object()` is `get_object(dbus_service_name, object_path)`. So, you can see from the above code snippet, `org.freedesktop.NetworkManager` is the service name and `/org/freedesktop/NetworkManager` is the object path. The object path is different for accessing different objects specified by the service. Here a proxy object referring to the `NetworkManager` is created. Now it is possible to access different properties of this object. For example, we can check whether the `NetworkManager` is in sleep or wake mode, or if it is connected to some network or not, as follows:

```
>>> print proxy_object.state() # To know the NM state
```

4

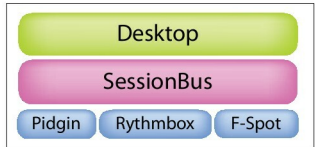


Figure 1: D-Bus SessionBus

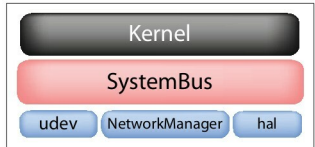


Figure 2: D-Bus SystemBus

The returned integer in the above example is called the `NM_STATE`. This corresponds to following states:

- `'NM_STATE_UNKNOWN = 0'` means the `NetworkManager` daemon is in an unknown state.
- `'NM_STATE_ASLEEP = 1'` means the `NetworkManager` daemon is asleep and all interfaces managed by it are inactive.
- `'NM_STATE_CONNECTING = 2'` means the `NetworkManager` daemon is connecting to a device.
- `'NM_STATE_CONNECTED = 3'` means the `NetworkManager` daemon is connected.
- `'NM_STATE_DISCONNECTED = 4'` means the `NetworkManager` daemon is disconnected.

Let's take a look at the following code:

```
>>> proxy_object.sleep() # Disable NetworkManager
>>> proxy_object.wake() # Enable NetworkManager
>>> proxy_object.GetDevices()
dbus.Array([dbus.ObjectPath('/org/freedesktop/Hal/devices/net_00_1c_23_fb_37_22'), dbus.ObjectPath('/org/freedesktop/Hal/devices/net_00_1c_bf_87_25_d2')], signature=dbus.Signature('o'))
```

You can see that the code lists objects of two network interfaces with MAC ID `00:1c:b8:87:25:d2` and `00:1c:23:fb:37:22` along with their HAL object paths. The `dbus.Array` element is a D-Bus object specific data type. We'll discuss more on D-Bus types in the later part of the article.

An object path can support any number of different interfaces. Before calling any method, you need to specify which interface you want to use. Interfaces are sub-objects that can be used to refer to a group of other objects to provide a higher level of abstraction on proxy objects and their exported methods. It provides a name-spacing mechanism. You can have a better understanding of the concepts of